

CGPO: Cognitive Graph Portfolio Optimizer for Dynamic Asset Allocation Using Graph Neural Networks and Reinforcement Learning

Vrushabh Jain, Geeta N. Brijwani, and Aditya Agarwal
Department of Computer Science, Kishinchand Chellaram College
HSNC University, Mumbai, India
Email: vrushabhjain2016@gmail.com

Abstract—Allocation models built on the Markowitz framework depend on sample covariance matrices that are estimated once and assumed stable, an assumption that fails precisely when markets turn turbulent. This paper presents the Cognitive Graph Portfolio Optimizer (CGPO), a system that treats an equity universe as a time-varying graph and learns allocation policies directly from that structure. At each step, assets become nodes carrying return, volatility, momentum, and relative strength features, while edges connect assets whose rolling correlation exceeds a threshold, with a k-nearest-neighbor fallback preserving connectivity. A two-layer graph convolutional network encodes this state for an advantage actor-critic agent whose actor outputs the concentration parameters of a Dirichlet distribution, so the resulting weights satisfy the portfolio simplex constraint by construction rather than by post-processing. The complete pipeline is deployed as a web application consisting of a Fast-API inference service on serverless GPU infrastructure and a browser dashboard that exposes the live asset graph, execution traces, and benchmark comparisons for United States and Indian equities. In a 104-day empirical study, graph density rose to 0.619 on the day of a synchronized 4.59 percent single-session decline across the tracked universe, evidence that the topology reacts to regime shifts. A seeded, reproducible ablation against a multilayer perceptron baseline finds the two architectures statistically indistinguishable at a 15-episode training budget, an outcome traced to the uniform prior of the Dirichlet policy and reported in full alongside its limitations.

Index Terms—portfolio optimization, graph neural networks, reinforcement learning, advantage actor-critic, Dirichlet policy, explainable AI, financial engineering

I. INTRODUCTION

A. Motivation

Capital allocation remains one of the oldest quantitative problems in finance, yet the tools most widely taught to solve it predate modern computing. Markowitz formulated portfolio selection as a trade-off between expected return and variance in 1952 [1], and the resulting mean-variance optimization (MVO) is still the default framing in textbooks and in the industry. The difficulty is not the elegance of the theory but its inputs. MVO requires an expected return vector and a covariance matrix, both estimated from history, and the optimizer amplifies whatever estimation error those inputs contain. Michaud described this behavior as “error maximization”: small perturbations in estimated means produce large, unstable swings in the optimal weights [2]. DeMiguel et al. later

showed that across many real datasets, a naive equal-weight rule is not reliably beaten by fourteen optimized extensions of the mean-variance model, largely because of this estimation fragility [3].

The second weakness is structural. The covariance matrix summarizes pairwise co-movement as a static block of numbers, but markets behave like networks whose wiring changes with conditions. Mantegna demonstrated in 1999 that correlation matrices of equity returns contain a meaningful hierarchical topology, with a cluster of stocks by sector and the cluster structure tightening during stress [4]. When a shock propagates from one sector to another, the relevant information is not any single correlation coefficient but the changing shape of the whole graph. Classical MVO has no mechanism to read that movement of the graph.

Two developments in machine learning make a different treatment possible. Geometric deep learning extends neural networks to non-Euclidean domains such as graphs [5], and graph convolutional networks (GCNs) give a practical and trainable operator to gather information in a neighborhood of nodes [6]. In parallel, deep reinforcement learning (RL) reframes allocation as sequential decision-making: rather than forecasting prices and optimizing afterwards, an agent learns a policy that maps the observed market state directly to portfolio weights [10], [11]. CGPO, the system described in this paper, combines the two: a GCN reads the market graph, and an actor-critic agent trained on that representation outputs the allocation.

B. Problem Statement

Formally, portfolio optimization allocates capital across n assets to maximize risk-adjusted return. The classical formulation is

$$\max_{\mathbf{w}} \quad \boldsymbol{\mu}^T \mathbf{w} - \frac{\lambda}{2} \mathbf{w}^T \boldsymbol{\Sigma} \mathbf{w} \quad (1)$$

subject to $\sum_{i=1}^n w_i = 1$ and $w_i \geq 0$, where $\mathbf{w}$ is the weight vector, $\boldsymbol{\mu}$ the expected return vector, $\boldsymbol{\Sigma}$ the covariance matrix, and λ a risk-aversion coefficient [1]. The assumptions embedded in (1) are stationary, approximately Gaussian returns and a covariance structure that remains valid over the holding period. Neither assumption survives contact with crisis regimes, during which return distributions exhibit fat tails and correlations

converge toward one [2], [4]. The problem this paper addresses is therefore twofold: to construct a market representation that updates its dependency structure continuously, and to learn an allocation policy on that representation which respects the simplex constraint natively.

C. Proposed Approach

CGPO models the asset universe as a dynamic graph $G_t = (V, E_t)$ whose edge set is recomputed at every step from rolling-window correlations. A two-layer GCN encoder [6], implemented in PyTorch Geometric [8], produces node embeddings that mix each asset’s own technical features with those of its correlated neighbors. An advantage actor-critic (A2C) agent [9] consumes these embeddings. Its actor head maps each node embedding to a concentration parameter of a Dirichlet distribution, so every sampled action is a valid point on the portfolio simplex without clipping or renormalization. Its critic head pools the whole graph into a scalar state-value estimate. The system is not a backtest script: it is deployed end to end, with GPU-served inference, authenticated APIs, and a dashboard that renders the live asset graph and the agent’s execution trace for human review.

D. Contributions

The paper makes four contributions. First, it describes a correlation-driven graph construction procedure with threshold filtering, a k-nearest-neighbor connectivity fallback, and correlation magnitudes retained as edge weights for the message-passing stage. Second, it formulates allocations as a Reinforcement Learning problem whose Dirichlet-parameterized policy satisfies the simplex constraint by construction. Third, it reports a deployed full-stack realization of the method covering both United States and Indian equity benchmarks, with explainability features motivated by a field survey of 36 prospective users. Fourth, it presents an empirical study comprising a topological analysis over 104 trading days and an ablation against a multilayer perceptron (MLP) baseline, reported in full, including the finding that at this training budget the graph encoder and the baseline remain statistically indistinguishable.

The remainder of the paper is organized as follows. Section II surveys related work. Section III details the proposed approach. Section IV covers implementation and deployment. Section V presents the experimental evaluation, and Section VI concludes.

II. LITERATURE SURVEY

A. Classical Portfolio Theory and Its Limits

Modern portfolio theory begins with Markowitz [1], whose mean-variance framework established diversification as a quantitative discipline. Its practical shortcomings are equally well documented. Michaud argued that the optimizer systematically overweights assets with overestimated returns and underestimated risk, making the output an amplifier of input noise [2]. DeMiguel, Garlappi, and Uppal evaluated fourteen sample-based extensions of the model across seven datasets

and found that none consistently outperformed the $1/N$ equal-weight heuristic out of sample, attributing the result to estimation error overwhelming the theoretical gains of optimization [3]. These critiques motivate methods that either avoid point estimates of expected returns entirely or adapt their risk model as conditions change. CGPO does both: it learns a policy rather than estimating μ , and it rebuilds its dependency graph at every step.

B. Network Models of Financial Markets

Treating markets as graphs predates deep learning. Mantegna constructed minimum spanning trees from stock correlation matrices and showed that the resulting hierarchy recovers economically meaningful sector groupings [4]. The contribution relevant here is the observation that topology itself carries regime information: during drawdowns the tree contracts as correlations rise. Geometric deep learning supplied the tools to learn from such structures. Bronstein et al. established the general framework for neural networks on non-Euclidean domains [5]; Kipf and Welling introduced the spectral-motivated GCN whose propagation rule CGPO adopts [6]; and Veličković et al. added attention-based aggregation in which edge importance is learned rather than fixed [7]. On the engineering side, PyTorch Geometric provides the sparse tensor machinery that makes per-step graph construction feasible at interactive latency [8].

C. Reinforcement Learning for Allocation

RL replaces the forecast-then-optimize pipeline with direct policy learning. Jiang, Xu, and Liang’s Ensemble of Identical Independent Evaluators was an early demonstration that a deep agent can manage a portfolio end to end, using a softmax output layer to produce weights [10]. Mnih et al. introduced the advantage actor-critic family on which CGPO’s training loop is built [9]. Hambly, Xu, and Yang provide a recent and thorough survey of RL across finance, noting open problems in sample efficiency and reward design that this paper encounters directly in Section V [11]. FinRL packaged these techniques into a reproducible open-source framework and established serverless backtesting practice [12]. Wang et al.’s DeepTrader is notable for conditioning the policy on a market-level risk indicator to balance return against drawdown [14], a concern CGPO addresses through volatility terms in its reward function.

D. Graph-Based Reinforcement Learning Allocators

The intersection of the two strands is recent. Soleymani and Paquet’s DeepPocket combined a graph convolutional feature extractor with an actor-critic agent and outperformed market indexes across several investment periods, including the COVID-19 crash [13]. Ekmekcioğlu and Pinar showed that incorporating graphical knowledge of asset relations improves stability relative to recurrent architectures for deep portfolio optimization [15]. Sun, Wei, and Yang attached a GraphSAGE extractor to a proximal policy optimization agent and reported improved risk-adjusted performance on Chinese index

TABLE I
SYSTEM TECHNOLOGY STACK

Layer	Technology	Deployment
Frontend	Next.js 16, React 19	Vercel
Backend	FastAPI, Python 3.11	Modal (NVIDIA T4)
ML framework	PyTorch, PyTorch Geometric	Serverless GPU
Data source	Yahoo Finance	Live REST API

constituents [16]. Most recently, Zhang proposed a graph-attention, multi-agent architecture with specialized risk and return agents, reporting a 1.34 Sharpe ratio on large United States universes [18]. These systems validate the architectural direction but are, without exception, offline research backtests. None is deployed as a live, latency-constrained service, none covers Indian equities, and none exposes its reasoning to the end user.

E. Explainability and the Research Gap

Adoption of learned allocators is constrained as much by trust as by accuracy. Černevičienė and Kabašinskas, reviewing 138 studies of explainable AI in finance, find that opacity is a recurring barrier to the use of deep models in regulated financial settings [17]. The gap CGPO targets follows from this survey of prior work: a graph-based RL allocator that is simultaneously (i) constraint-satisfying by construction, (ii) deployed end to end under real latency and reliability requirements, and (iii) transparent enough that a human can audit every step from data fetch to weight vector. To our knowledge no prior system meets all three conditions at once.

III. PROPOSED APPROACH

A. System Overview

CGPO is organized as a decoupled client-server pipeline (Fig. 1). A data layer fetches live price histories and news; an intelligence core converts prices into a graph, encodes the graph, and produces portfolio weights; a serving layer exposes these functions behind authenticated HTTP endpoints; and a browser client renders the results. Table I summarizes the technology choices. Splitting the frontend from the inference engine keeps the interactive surface responsive while tensor computation runs on dedicated cloud GPUs, a deployment pattern consistent with the service-oriented practice established by open RL trading frameworks [12].

B. Design Requirements from a Field Survey

Before development, a survey of 36 prospective users, spanning finance professionals, technology and finance students, and retail investors, was conducted to ground the design in observed demand rather than assumption. Three findings shaped the system. A clear majority, 22 of 36 respondents (61 percent), preferred a network-map visualization of systemic risk over tabular formats, which motivated rendering the live asset graph in the client. A still larger share, 32 of 36 (89 percent), wanted decision support with human approval rather than fully automated execution, so CGPO is deliberately

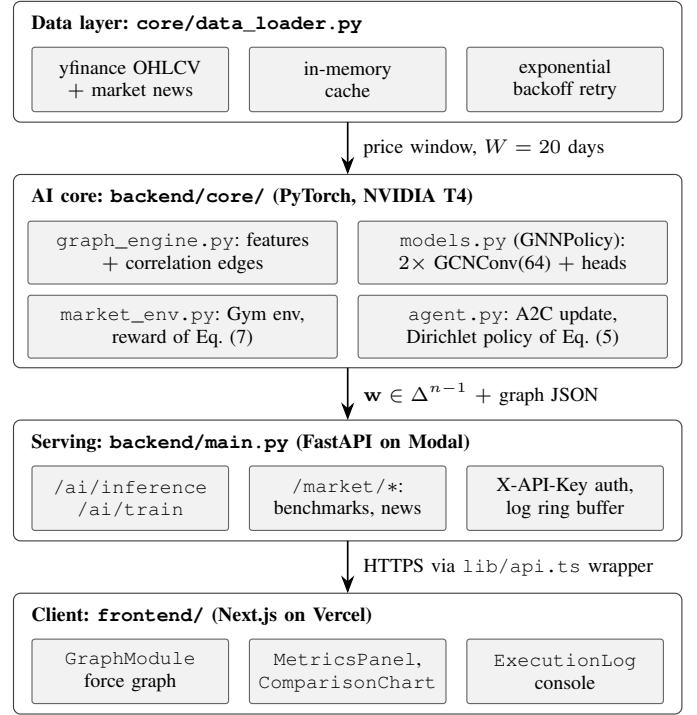


Fig. 1. CGPO deployment architecture mapped onto the repository layout. Market data flows through the graph construction and GNN-RL modules of `backend/core/`, is served through authenticated Fast-API endpoints defined in `main.py`, and reaches the browser components under `frontend/`. The service-oriented split follows deployment practice in open RL trading platforms [12]; the encoder design follows [6], [8].

advisory: it computes weights and presents them for review, never trading autonomously. Finally, 28 of 36 (78 percent) required real-time inference rather than end-of-day reports, which ruled out batch processing and dictated the serverless GPU backend. The third finding aligns with the trust concerns documented in the explainable-AI literature [17].

C. Market Graph Construction

At each timestep t the universe of n assets is encoded as a weighted graph $G_t = (V, E_t)$ (Fig. 2).

1) *Node features*: Each asset i carries a characteristic vector $\mathbf{x}_i \in \mathbb{R}^4$ computed over a lookback window $W = 20$ trading days: the single-step percentage return, the realized volatility (standard deviation of returns over W), the period momentum (cumulative return over W) and the standard 14-period Relative Strength Index [19] rescaled to $[0, 1]$, with a neutral value of 0.5 substituted when there is insufficient history. The window length of one trading month balances the response against noise.

2) *Edge construction*: Edges are derived from the Pearson correlation matrix $\rho \in \mathbb{R}^{n \times n}$ of returns over the rolling window,

$$\rho_{ij} = \frac{\text{cov}(r_i, r_j)}{\sigma_i \sigma_j}, \quad (2)$$

with an edge placed between assets i and j whenever $|\rho_{ij}| > \tau$. All experiments reported here use $\tau = 0.5$; the deployed ser-

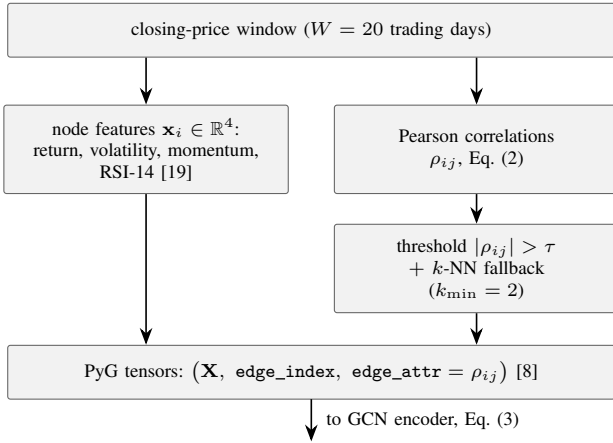


Fig. 2. Per-step graph construction as implemented in `graph_engine.py`. Technical features populate the nodes while thresholded rolling correlations define the edge set; signed correlations are stored as edge attributes and their magnitudes weight message passing in the GCN encoder [6], [8].

vice exposes the threshold as configuration. Pearson correlation measures only linear dependence, but recomputing it on a rolling basis converts a static statistic into a dynamic topology at negligible computational cost, which is the property CGPO requires. The threshold suppresses weak, noisy links; the trade-off is that during periods of low cross-sectional correlation a node may lose all its edges. To prevent isolated nodes, a k -nearest-neighbor fallback guarantees each node at least $k_{\min} = 2$ edges, chosen by descending correlation magnitude. Signed correlations ρ_{ij} are stored as edge attributes rather than discarded after thresholding, and their magnitudes $|\rho_{ij}|$ serve as edge weights on the encoder, influencing the neighborhood scaling by the strength of the dependence.

D. Graph Neural Encoder

The encoder stacks two graph convolutional layers [6] with hidden dimension 64 and dropout $p = 0.1$ between them. The layer-wise propagation rule is

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}\right), \quad (3)$$

where $\tilde{A} = A + I$ is the adjacency matrix with self-loops, $\tilde{D}$ its degree matrix, $W^{(l)}$ the trainable weights, and σ a nonlinearity. Message passing is weighted by the stored correlation magnitudes, so a neighbor correlated at 0.9 contributes more to an asset's embedding than one correlated at 0.55. Two layers give each node a two-hop receptive field: an asset's embedding reflects not only its direct correlates but the broader cluster it belongs to, which is the structural signal Mantegna identified in market trees [4].

E. Reinforcement Learning Formulation

Allocation is cast as a Markov decision process. The state s_t is the graph G_t with its node features; the action a_t is a weight vector on the simplex $\Delta^{n-1} = \{\mathbf{w} \in \mathbb{R}^n : w_i \geq 0, \sum_i w_i = 1\}$; the transition is the market's evolution to $t + 1$.

1) *Dirichlet policy*: The actor head maps each node embedding to a logit z_i and transforms it into a Dirichlet concentration parameter

$$\alpha_i = \text{softplus}(z_i) + 1, \quad (4)$$

which guarantees $\alpha_i > 1$ and hence a unimodal density over the interior of the simplex. The policy is the Dirichlet distribution

$$\pi(\mathbf{w} | s_t) = \frac{1}{B(\boldsymbol{\alpha})} \prod_{i=1}^n w_i^{\alpha_i - 1}, \quad (5)$$

where $B(\boldsymbol{\alpha})$ is the multivariate beta function and the support is exactly Δ^{n-1} . Every sample is therefore a feasible portfolio without clipping or projection; the environment retains a defensive renormalization for arbitrary inputs to its Box action space, but for the agent's Dirichlet outputs this operation is the identity. During training the agent samples from (5) for exploration; at inference it acts deterministically through the distribution mean $w_i = \alpha_i / \sum_j \alpha_j$. This contrasts with softmax-output designs [10], which produce a point on the simplex but not a distribution over it, and therefore cannot trade off exploration against concentration in the same way.

2) *Critic and losses*: The critic applies global mean pooling over all node embeddings to form a market-level state representation, projected to a scalar estimate $V(s_t)$. Training follows the A2C scheme [9], minimizing

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + 0.5 \mathcal{L}_{\text{value}} - \beta S, \quad (6)$$

where $\mathcal{L}_{\text{policy}} = -\mathbb{E}[\log \pi(a_t | s_t) A_t]$ uses the advantage estimate $A_t = U_t - V(s_t)$, with U_t the return observed from state s_t onward (written U_t to keep it distinct from both the graph G_t and the per-step reward R_t defined below), $\mathcal{L}_{\text{value}}$ is the mean squared error of the critic, and S is the policy entropy with coefficient $\beta = 0.01$, included to delay premature convergence to concentrated allocations. Discounted returns are normalized per episode before the advantage computation, gradients are clipped to a maximum norm of 0.5, and updates producing non-finite losses are skipped. Optimization uses Adam with learning rate 10^{-3} and discount factor $\gamma = 0.99$.

3) *Reward design*: The per-step reward balances outperformance against stability:

$$R_t = 100(r_p - r_b) - 50|r_p| - 10\sigma_{20} + 10\bar{e}_5 \mathbf{1}_{\{\bar{e}_5 > 0\}}, \quad (7)$$

where r_p is the portfolio return, r_b the benchmark return, σ_{20} the 20-day rolling volatility of the portfolio, and $\bar{e}_5$ the 5-day moving average of excess returns; the volatility and consistency terms activate once sufficient return history has accumulated. The first term rewards beating the benchmark. The second penalizes large single-step swings in either direction, discouraging high-turnover churning. The third enforces longer-horizon stability, in the spirit of the risk-return balancing argued for by DeepTrader [14]. The final term pays a consistency bonus only while the excess-return average is positive. The coefficients were tuned empirically; their sensitivity is acknowledged as a limitation in Section V.

IV. IMPLEMENTATION AND DEPLOYMENT

A. Data Pipeline

Market data is fetched from the Yahoo Finance REST interface with parallel per-ticker requests. Because public financial APIs drop connections and rate-limit aggressively, every fetch is wrapped in an exponential backoff decorator (base delay 1 s, doubling per attempt, three attempts) with a local cache in front, so a single failed call degrades gracefully instead of aborting an inference cycle. Missing values are forward-filled before feature computation, and delisted or invalid tickers are dropped from the node set rather than propagated as NaN features.

B. Serving Stack

The inference engine runs as a FastAPI application deployed on Modal serverless infrastructure with NVIDIA T4 GPUs, scaling to zero between requests so that GPU cost is incurred only during active use. The frontend is a Next.js application served from Vercel. All non-public endpoints require an API key delivered via request header; the deployment additionally sets restrictive CORS rules, standard hardening headers, and a 15-second client-side timeout that aborts hanging requests before they hold a GPU. End-to-end inference, from request arrival through data fetch, graph construction, GNN forward pass, and response, completes in under two seconds in production tests, meeting the real-time requirement established by the user survey.

C. Explainability Surface

Three interface elements address the opacity concern documented in the XAI literature [17]. The dashboard renders the live asset graph with force-directed layout, so the user sees the same topology the encoder sees, including which assets are currently hubs. An execution log, kept server-side in a 200-entry ring buffer and polled by the client, streams each backend step (data fetch, graph construction, forward pass, weight output) as it happens, making the inference pipeline auditable rather than a black box. Finally, allocation outputs are presented alongside benchmark comparisons and risk metrics for human approval, consistent with the advisory posture chosen in Section III-B.

D. Robustness Validation

Beyond unit tests, the system was validated against edge cases chosen to break naive implementations (Table II). Each scenario exercises a specific failure mode of the graph-RL pipeline: degenerate correlation structures, zero-volatility windows that would divide by zero in (7), and malformed user input.

V. EXPERIMENTAL EVALUATION

A. Setup

The empirical study covers a universe of seven large-capitalization United States equities (NVDA, GOOG, AAPL, AMZN, MSFT, META, TSLA) over the 104 trading days ending in late December 2025, drawn from a date range pinned

TABLE II
SYSTEM INTEGRATION AND EDGE-CASE TESTING

ID	Scenario	Verified behaviour
TC01	Invalid or delisted ticker	Loader drops the node and continues
TC02	Zero-volatility window	Reward computation avoids division by zero
TC03	Disconnected correlation matrix	k -NN fallback restores $k_{\min} = 2$ connectivity
TC04	Action-space validation	Output weights sum to exactly 1.0
TC05	End-to-end latency	Inference completes in under 2.0s

to June through December 2025. The initial portfolio value is \$10,000, and the SPY exchange-traded fund serves as the S&P 500 benchmark proxy. Both the GNN agent and the MLP baseline were trained from scratch for 15 episodes on identical historical data with all random number generators seeded identically (seed 42). A budget constrained by available compute; the consequences of this constraint are analyzed in Section V-D. At the evaluation time both agents act deterministically through the Dirichlet mean. Sharpe ratios are annualized as the mean of daily returns over their standard deviation, scaled by $\sqrt{252}$, with no risk-free adjustment. The MLP baseline receives the same node features as the GNN, flattened into a single vector, so the comparison isolates the value of the graph structure itself rather than differences in input information. The evaluation script, with the pinned window and seed, is committed to the project repository so that every figure in Table IV can be regenerated with a single command.

B. Topological Analysis

A central claim of this paper is that graph structure carries market information beyond individual price series. Two observations from a 104-trading-day observation window, logged by the deployed system in the second half of 2025, support this. First, degree centrality leadership rotates: Table III and Fig. 3 report how often each asset was the most-connected node in the daily graph. NVDA led on 25 of 104 days, but no asset dominated, and leadership shifts tracked sector news flow. The table lists the five most frequent leaders; the two remaining sessions were led by other members of the seven-asset universe, which is why the counts sum to 102 rather than 104. An asset's centrality measures how many strong correlates it currently has, so the most central node is the one whose movements are most representative of the cluster, information that is invisible to any per-asset indicator.

Second, graph density acts as a systemic-risk gauge. Density peaked at 0.619 on October 30, 2025, the same session as a synchronized single-day decline of 4.59 percent across the tracked universe, when pairwise correlations converged and the thresholded graph approached a clique. This is the dynamic counterpart of the crisis-time correlation tightening described in the econophysics literature [4]: the topology registered the regime shift on the day it occurred, without any single asset's features being unusual in isolation.

TABLE III
DEGREE CENTRALITY LEADERSHIP OVER 104 TRADING DAYS

Ticker	Days as most central	Share
NVDA	25	24.0%
GOOG	21	20.2%
AAPL	21	20.2%
AMZN	19	18.3%
MSFT	16	15.4%

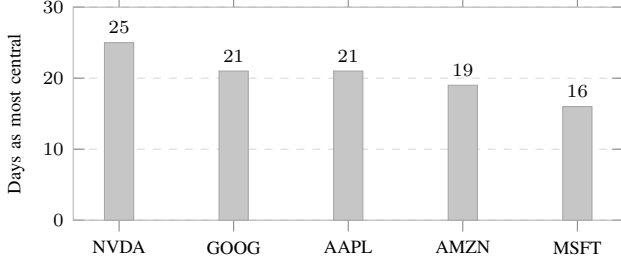


Fig. 3. Degree-centrality leadership counts across the 104-day observation window. Centrality is computed on the thresholded correlation graph of Section III-C; rotation of the most-connected node reflects shifting cluster structure of the kind documented in correlation-network studies [4].

TABLE IV
ABLATION STUDY OVER 104 TRADING DAYS: GNN VERSUS MLP
BASELINE VERSUS SPY

Metric	GNN (GCN)	MLP	SPY
Cumulative return	17.56%	17.49%	9.47%
Annualized Sharpe	1.95	1.94	1.70
Maximum drawdown	8.9%	8.9%	5.1%
Final value	\$11,755.77	\$11,748.53	\$10,947.14
Mean maximum weight	0.143	0.148	—

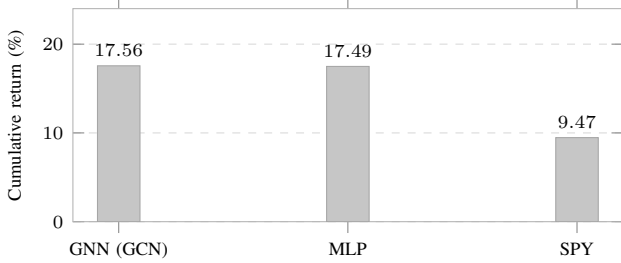


Fig. 4. Cumulative returns over the pinned 104-day evaluation window for the GNN agent, the flattened MLP baseline, and the SPY benchmark, under an identical seeded 15-episode training protocol. Both learned agents use the A2C objective of Eq. (6) [9].

C. Ablation: Graph Encoder versus MLP Baseline

Table IV and Fig. 4 report the ablation. Under the 15-episode budget the two learned agents finished within seven basis points of each other: 17.56 percent cumulative return for the GNN against 17.49 percent for the MLP, with annualized Sharpe ratios of 1.95 and 1.94. Both outperformed the benchmark’s 9.47 percent return and 1.70 Sharpe ratio over the identical window, at the cost of a deeper maximum drawdown (8.9 against 5.1 percent).

D. Discussion

We report this result as it stands rather than tuning until the preferred architecture wins. The most informative reading is not that either architecture failed, but that at this budget the two are statistically indistinguishable, and the weight statistics explain why.

The mean maximum portfolio weight makes the under-training directly visible. The GNN averaged 0.143 and the MLP 0.148, against $1/7 \approx 0.143$ for the uniform allocation: neither agent moved meaningfully away from uniform weights. This is a consequence of the policy parameterization in (4), which guarantees $\alpha_i > 1$ and therefore places the Dirichlet mean at the uniform portfolio from initialization; fifteen episodes of gradient steps are too few to displace the concentration parameters from that prior, and the trendless per-episode reward traces of both arms corroborate this. Fifteen episodes is far below what graph-based RL agents require for convergence: the GNN must learn both an encoding of graph structure and a policy on top of it, a harder optimization problem than the MLP’s direct feature-to-action mapping, and the literature on graph RL allocators reports convergence over hundreds to thousands of episodes [13], [16]. The comparison therefore measures early-training behaviour, not asymptotic capability, and any structural advantage of the encoder can only emerge at larger budgets.

The benchmark comparison should be read with equal care. Both agents beat SPY by roughly eight percentage points, but a near-uniform portfolio of seven mega-capitalization technology stocks is a sector bet, not evidence of learned skill: the sector outperformed the broad market in this window, and the agents paid for the tilt with a deeper maximum drawdown (8.9 against 5.1 percent); the synchronized October 30 decline identified by the density analysis of Section V-B illustrates the mechanism. Holding the cluster uniformly approximates holding the cluster itself. This observation is consistent with the equal-weight literature [3] and argues for cross-sector universes, where graph-aware diversification can separate genuinely independent risk sources, as the setting where the encoder’s structural information should matter most.

Beyond these qualifications, the evaluation has limitations we state explicitly: a seven-asset, single-sector universe; a single random seed without variance estimates; an in-sample protocol in which the training episodes iterate over the same window used for evaluation, with no walk-forward split; no transaction-cost or slippage model; hand-tuned reward coefficients in (7); and Pearson correlation as the only dependence measure. Each is addressable within the existing architecture.

VI. CONCLUSION AND FUTURE WORK

This paper argued that the persistent weaknesses of mean-variance allocation, estimation fragility and structural blindness, are best addressed by changing the representation rather than patching the estimator. CGPO contributes a complete realization of that argument: equity markets modeled as dynamic correlation graphs, encoded by a weighted GCN, and acted upon by an A2C agent whose Dirichlet policy

guarantees that every output is a feasible portfolio. The system runs as a complete deployed service with sub-two-second inference, authenticated serving, and an explainability surface designed from user-survey evidence. Empirically, the graph representation demonstrated regime sensitivity, with density spiking on the day of a synchronized drawdown, while the controlled ablation showed that under a 15-episode budget the graph agent and a flattened baseline remain statistically indistinguishable, both holding near-uniform allocations, a finding we report transparently and attribute to under-training relative to the convergence demands of graph reinforcement learning.

Future work proceeds on three fronts. Scaling training to the order of 10^3 episodes with multi-seed evaluation will test whether the encoder's structural advantages materialize asymptotically. Replacing the fixed correlation weighting with learned attention [7] and extending the universe across sectors, asset classes, and the Indian market already supported by the platform will exercise the diversification argument of Section V-D. Finally, appending news-sentiment features to the node vectors, for which the data pipeline already collects signals, would let the graph carry qualitative as well as statistical dependence.

REFERENCES

- [1] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
- [2] R. O. Michaud, "The Markowitz optimization enigma: Is 'optimized' optimal?" *Financial Analysts Journal*, vol. 45, no. 1, pp. 31–42, 1989.
- [3] V. DeMiguel, L. Garlappi, and R. Uppal, "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?" *The Review of Financial Studies*, vol. 22, no. 5, pp. 1915–1953, 2009.
- [4] R. N. Mantegna, "Hierarchical structure in financial markets," *The European Physical Journal B*, vol. 11, no. 1, pp. 193–197, 1999.
- [5] M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst, "Geometric deep learning: Going beyond Euclidean data," *IEEE Signal Processing Magazine*, vol. 34, no. 4, pp. 18–42, 2017.
- [6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.
- [7] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.
- [8] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [9] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, vol. 48, 2016, pp. 1928–1937.
- [10] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," *arXiv preprint arXiv:1706.10059*, 2017.
- [11] B. Hambly, R. Xu, and H. Yang, "Recent advances in reinforcement learning in finance," *Mathematical Finance*, vol. 33, no. 3, pp. 437–503, 2023.
- [12] X.-Y. Liu, H. Yang, J. Gao, and C. D. Wang, "FinRL: Deep reinforcement learning framework to automate trading in quantitative finance," in *Proc. 2nd ACM Int. Conf. AI in Finance (ICAIF)*, 2021.
- [13] F. Soleymani and E. Paquet, "Deep graph convolutional reinforcement learning for financial portfolio management – DeepPocket," *Expert Systems with Applications*, vol. 182, art. 115127, 2021.
- [14] Z. Wang, B. Huang, S. Tu, K. Zhang, and L. Xu, "DeepTrader: A deep reinforcement learning approach for risk-return balanced portfolio management with market conditions embedding," in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, no. 1, 2021, pp. 643–650.
- [15] Ö. Ekmekcioğlu and M. Ç. Pinar, "Graph neural networks for deep portfolio optimization," *Neural Computing and Applications*, vol. 35, no. 28, pp. 20663–20674, 2023.
- [16] Q. Sun, X. Wei, and X. Yang, "GraphSAGE with deep reinforcement learning for financial portfolio optimization," *Expert Systems with Applications*, vol. 238, art. 122027, 2024.
- [17] J. Černevičienė and A. Kabašinskas, "Explainable artificial intelligence (XAI) in finance: A systematic literature review," *Artificial Intelligence Review*, vol. 57, art. 216, 2024.
- [18] B. Zhang, "Graph attention-based heterogeneous multi-agent deep reinforcement learning for adaptive portfolio optimization," *Scientific Reports*, vol. 15, 2025.
- [19] J. W. Wilder, *New Concepts in Technical Trading Systems*. Greensboro, NC: Trend Research, 1978.